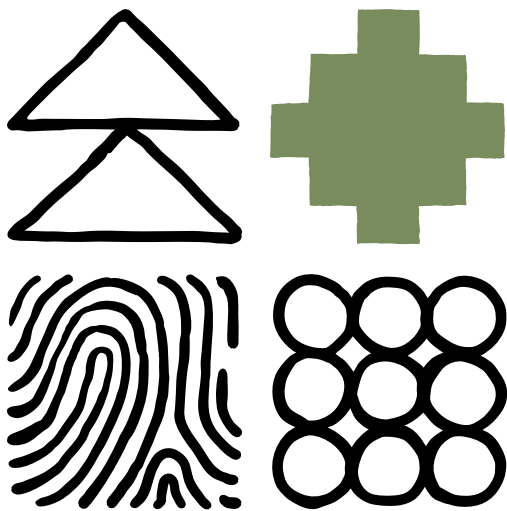


Engineering at Anthropic

Anthropic 的工程



Quantifying infrastructure noise in agentic coding evals

量化智能编码评估中的基础设施噪声

Infrastructure configuration can swing agentic coding benchmarks by several percentage points—sometimes more than the leaderboard gap between top models.

基础设施配置可能会使代理编码基准的表现波动好几个百分点——有时甚至超过顶尖模型之间的领先差距。

Agentic coding benchmarks like SWE-bench and Terminal-Bench are commonly used to compare the software engineering capabilities of frontier models—with top spots on leaderboards often separated by just a few percentage points. These scores are often treated as precise measurements of relative model capability and increasingly inform decisions about which models to deploy. However, we've found that infrastructure configuration alone can produce differences that exceed

those margins. In internal experiments, the gap between the most- and least-resourced setups on Terminal-Bench 2.0 was 6 percentage points ($p < 0.01$).

代理编码基准测试如 SWE-bench 和 Terminal-Bench 常用于比较前沿模型的软件工程能力——排行榜上的顶尖名次往往仅相差几个百分点。这些评分通常被视为模型相对能力的精确测量，并越来越多地指导部署模型的决策。然而，我们发现仅基础设施配置就可能产生超出这些界限的差异。内部实验显示，终端-工作台 2.0 中资源最有限的设置差距为 6 个百分点 ($p < 0.01$)。

Static benchmarks score a model's output directly—the runtime environment doesn't factor into the result. Agentic coding evals are different: models are given a full environment where they write programs, run tests, install dependencies, and iterate over multiple turns. The runtime is no longer a passive container, but an integral component of the problem-solving process. Two agents with different resource budgets and time limits aren't taking the same test.

静态基准直接对模型输出进行评分——运行环境不影响结果。代理编码评估则不同：模型会有一个完整的环境，编写程序、运行测试、安装依赖关系，并多次迭代。运行时不再是一个被动的容器，而是问题解决过程的不可或缺的组成部分。两个资源预算和时间限制不同的特工参加的测试并不相同。

Eval developers have begun accounting for this. Terminal-Bench 2.0, for instance, specifies recommended CPU and RAM on a per-task basis in their latest 2.0 release. However, specifying resources isn't the same as enforcing them consistently. Moreover, we discovered that enforcement methodology can change what the benchmark ends up actually measuring.

评估开发者已经开始考虑这一点。例如，Terminal-Bench 2.0 在其最新版本 2.0 中，按任务指定推荐的 CPU 和内存。然而，指定资源并不等同于持续执行它们。此外，我们发现执法方法可以改变基准实际衡量的内容。

How we got here 我们是如何走到这一步的

We run Terminal-Bench 2.0 on a Google Kubernetes Engine cluster. While calibrating the setup, we noticed our scores didn't match the benchmark's official leaderboard, and infra error rates were surprisingly high: as many as 6% of tasks were failing because of pod errors, most of which were unrelated to the model's ability to solve the tasks.

我们在 Google Kubernetes Engine 集群上运行 Terminal-Bench 2.0。在校准设置时，我们发现分数与基准测试的官方排行榜不符，基础设施错误率也出乎意料地高：多达 6% 的任务因 Pod 错误失败，而大多数错误与模型解决任务的能力无关。

The discrepancy in scores came down to enforcement. Our Kubernetes implementation treated the per-task resource specs as both a floor and a hard ceiling: each container was guaranteed the specified resources but killed the moment it exceeded them. Container runtimes enforce resources via two separate parameters: a guaranteed allocation—the resources reserved up front—and a hard limit at which the container is killed. When these are set to the same value, there's zero headroom for transient spikes: a momentary memory fluctuation can OOM-kill a container that would otherwise have succeeded. To account for this, Terminal-Bench's leaderboard uses a different sandboxing provider, whose implementation is more lenient, allowing temporary overallocation without terminating the container in order to favor infrastructural stability.

分数差异归结于执法。我们的 Kubernetes 实现将每个任务的资源规格既视为下限，也作为硬性上限：每个容器保证获得指定资源，但一旦超过即被终止。容器运行时通过两个独立参数强制资源：保证分配——即预留的资源——以及容器被杀死的硬性限制。当这些值设置为相同时，暂时尖峰就没有余量：一瞬间的内存波动可能会让本来可能成功的容器被 OOM 杀死。为此，Terminal-Bench 的排行榜采用了不同的沙箱服务商，其实施更为宽松，允许临时超分配而无需终止容器，以有利于基础设施稳定性。

This finding raised a larger question: how much does resource configuration impact evaluation scores?

这一发现引出了一个更大的问题：资源配置评估评分对评分的影响有多大？

To quantify the effect of the scaffold, we ran Terminal-Bench 2.0 across six resource configurations, from strict enforcement of the per-task specs (1x), having them act as both floor and ceiling, to completely uncapped. Everything else stayed constant: same Claude model, same harness, same task set.

为了量化脚手架的影响，我们将 Terminal-Bench 2.0 应用于六种资源配置，从严格执行每个任务的规格（1 倍），让它们既作为下限又作为上限，到完全无上限。其他一切都保持不变：同款 Claude 型号，同款背带，同样的任务集。

In our experiments, success rates increased with resource headroom. This was primarily driven by infra error rates dropping monotonically at each step, going from 5.8% at strict enforcement to 0.5% when uncapped. The drop between strict enforcement to 3x headroom (5.8% to 2.1%) was significant at $p < 0.001$. With more

headroom, fewer containers get killed for exceeding their allocation.

在我们的实验中，随着资源余量增加，成功率也随之提高。这主要是由于每一步的基础设施错误率单调下降，严格执行时的 5.8% 降至无上限时的 0.5%。严格执法到 3 倍净量的下降（5.8% 降至 2.1%）在 $p < 0.001$ 时显著。有了更大的头部空间，超过分配量而被杀的集装箱也更少。

From 1x through 3x, success scores fluctuate within the margins of noise ($p=0.40$). Most of the tasks that were crashing at 1x would have failed regardless—which is something that we observed in the data. The agent explores, hits a resource wall, and gets preempted, but it was never on a path to a correct solution.

从 1x 到 3x，成功分数在噪声范围内波动（ $p=0.40$ ）。大多数以 1 倍速度崩溃的任务无论如何都会失败——这也是我们在数据中观察到的。代理探索时遇到资源瓶颈，被抢先行动，但从未走上正确解决方案的路径。

Starting around 3x, however, this trend changes: success rates climb faster than infra errors decline.

但从大约 3 倍开始，这一趋势发生变化：成功率上升速度快于基础设施误差下降。

Between 3x to uncapped, infra errors drop an additional 1.6 percentage points, while success jumps almost 4 percentage points. The extra resources enable the agent to try approaches that only work with generous allocations, such as pulling in large dependencies, spawning expensive subprocesses, and running memory-intensive test suites. At uncapped resources, the total lift over 1x is +6 percentage points ($p < 0.01$). At the margins, tasks like `rstan-to-pystan` and `compile-compcert` significantly improve their success rates when getting memory headroom.

在 3 倍到无上限之间，基础设施误差会额外下降 1.6 个百分点，而成功率则跃升近 4 个百分点。额外的资源使代理能够尝试只在宽松分配下运行的方法，比如拉入大型依赖、生成昂贵的子进程以及运行内存密集型测试套件。在无上限资源下，1 倍以上的总提升为 +6 个百分点（ $p < 0.01$ ）。在边际，像 `rstan-to-pystan` 和 `编译-计算证书` 这样的任务在获得内存余量时显著提高了成功率。

How this affects measurement

这对测量的影响

Up to roughly 3x Terminal-Bench specs, the additional resources fix infrastructure reliability problems, namely transient resource spikes. The sandboxing provider used by the Terminal-Bench maintainers is implicitly doing this behind the scenes;

the eval gets more stable without getting easier.

在大约3倍终端-工作台规格下，新增资源解决了基础设施可靠性问题，尤其是暂时的资源峰值。终端工作台维护者使用的沙箱提供商实际上在幕后默默地进行这些操作；评估会变得更稳定，但不会变得更容易。

Above the 3x mark, however, additional resources start actively helping the agent solve problems it couldn't solve before, which shows that limits can actually change what the eval measures. Tight limits inadvertently reward very efficient strategies, while generous limits are more forgiving and reward agents that can better exploit all available resources.

然而，超过3倍后，额外资源会主动帮助代理人解决之前无法解决的问题，这表明极限实际上会改变评估所衡量的内容。严格的限制无意中奖励非常高效的策略，而宽松的限制则更宽容，奖励能够更好地利用所有可用资源的代理人。

An agent that writes lean, efficient code very fast will do well under tight constraints. An agent that brute-forces solutions with heavyweight tools will do well under generous ones. Both are legitimate things to test, but collapsing them into a single score without specifying the resource configuration makes the differences—and real-world generalizability—hard to interpret.

一个能快速写出精简高效代码的代理，在严格的约束下表现良好。一个用重型工具蛮力强行解决问题的代理，在慷慨工具下会表现良好。这两者都是合理的测试对象，但如果将它们归纳为一个评分而不具体说明资源配置，会使差异和现实世界的普遍性难以解读。

On `bn-fit-modify`, a Terminal-Bench task requiring Bayesian network fitting, some models' first move is to install the standard Python data science stack: `pandas`, `networkx`, `scikit-learn`, and all their toolchain. Under generous limits, this works. Under tight ones, the pod runs out of memory during installation, before the agent writes a single line of solution code. A leaner strategy exists (implementing the math from scratch using only the standard library), and some models do default to it. Others don't. Different models have different default approaches, and the resource configuration determines which of those approaches happen to succeed. We replicated the core finding across different Anthropic models. The direction of the effect was consistent, while the magnitude varied. The same trends seem to hold on models other than Claude, but we haven't rigorously

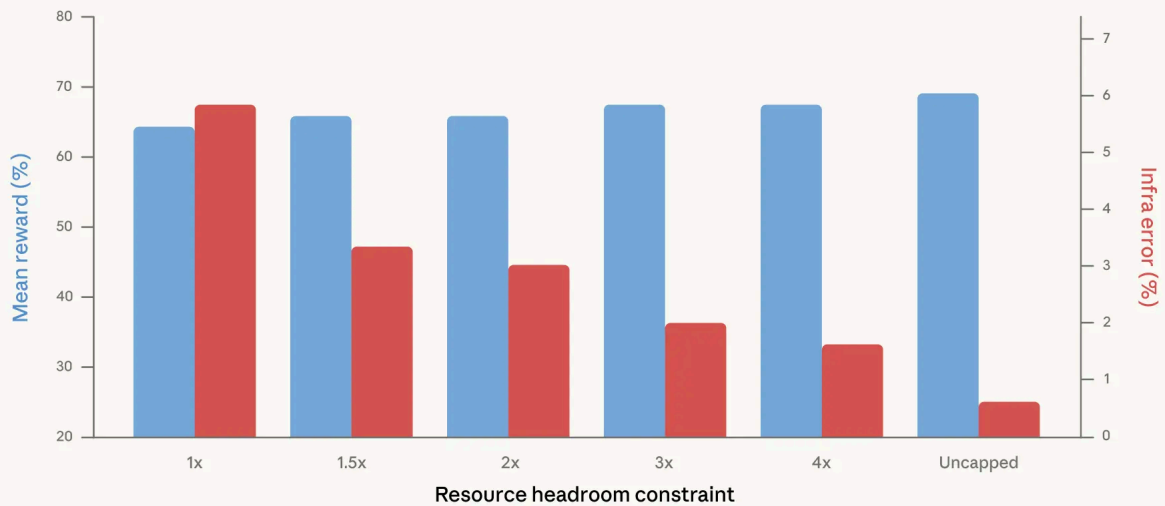
tested them.

在 `bn-fit-modify`（一个需要贝叶斯网络拟合的终端-工作台任务）中，一些模型的第一步是安装标准的 Python 数据科学堆栈：`pandas`、`networkx`、`scikit-learn` 及其所有工具链。在宽松的限制下，这种做法是可行的。在紧密的存储器中，安装时 pod 内存耗尽，代理尚未写出一行解决方案代码。有一种更精简的策略（从零开始仅用标准库实现数学），有些模型默认采用这种方法。而有些则不然。不同模型有不同的默认方法，资源配置决定了哪些方法成功。我们在不同的人类模型中重复了核心发现。效应的方向是一致的，而幅度则有所不同。除了 Claude 之外，其他模型似乎也存在类似趋势，但我们还没有严格测试过。

We also tested whether this pattern holds on evals outside Terminal-Bench by running a crossover experiment on SWE-bench. We varied the total available RAM up to 5x the baseline across 227 problems with 10 samples each. The same effect held, though the magnitude was smaller: Scores again increased monotonically with RAM, but were only 1.54 percentage points higher at 5x than 1x. SWE-bench tasks are less resource-intensive, so a smaller effect is expected, but it shows resource allocation isn't neutral there either.

我们还通过在 SWE 实验台上运行交叉实验，测试了该模式在终端-工作台外的评估中是否成立。我们在 227 个问题中，每个问题 10 个样本，将总可用内存变化为基线的 5 倍。同样的效果依然存在，但幅度较小：分数再次单调地随 RAM 增长，但仅高出 1.54 个百分点，达到 5 倍。软件工程实验台任务资源消耗较低，因此预期影响较小，但这也表明资源分配在该领域并非中立。

Success rate vs infra error rate



Other sources of variance

其他方差来源

Resource allocation isn't the only hidden variable. In certain configurations, time limits too start playing a role.

资源分配并不是唯一的隐藏变量。在某些配置中，时间限制也开始发挥作用。

In principle, every element of the evaluation setup can influence the final score, from the cluster health to the hardware specs, from the concurrency level to even egress bandwidth. Agentic evals are end-to-end system tests by construction, and any component of that system can act as a confounder. We have observed anecdotally, for instance, that pass rates fluctuate with time of day, likely because API latency varies with traffic patterns and incidents. We have not formally quantified this effect, but it illustrates a larger point: the boundary between "model capability" and "infrastructure behavior" is blurrier than a single benchmark score suggests. A model provider can shield its eval infrastructure from this by

dedicating hardware, but external evaluators can't easily do the same.

原则上，评估设置的每一个元素都会影响最终得分，从集群健康状况到硬件规格，从并发水平到出口带宽。代理评估从结构上讲就是端到端的系统测试，系统的任何组成部分都可能成为混杂因素。例如，我们观察到，通过率会随时间变化，这很可能是因为 API 延迟会随着流量模式和事件而变化。我们尚未正式量化这一效应，但它说明了一个更重要的观点：“模型能力”与“基础设施行为”之间的界限比单一基准评分所暗示的要模糊。模型提供商可以通过专用硬件保护其评估基础设施免受此类影响，但外部评估者却难以做到同样的效果。

Public benchmarks are typically meant to measure pure model capabilities, but in practice they risk conflating them with infrastructure quirks. Sometimes this may be desirable, as it enables end-to-end testing of the entire stack, but more often it's not. For coding evals meant to be shared publicly, running at multiple times and on multiple days would help average out the noise.

公开基准通常旨在衡量纯模型能力，但实际上它们有可能将其与基础设施特性混淆。有时这可能是理想的，因为它支持对整个栈的端到端测试，但更多时候并非如此。对于应公开共享的编码评估，多次运行、多天运行有助于平均噪声。

What we recommend 我们的推荐

The ideal scenario is to run each eval under the exact same hardware conditions—both the scaffold running the eval and the inference stack—as it would ensure perfect reproducibility across the board. However, this may not always be practical.

理想的情况是在完全相同的硬件条件下运行每次评估——既是执行评估的支架，也是推理栈——以确保所有系统的完美复现性。然而，这并不总是实际可行。

Given how container runtimes actually enforce resources—via a guaranteed allocation and a separate hard kill threshold—we recommend that evals specify both parameters per task, not a single pinned value. A single exact spec sets the guaranteed allocation equal to the kill threshold, leaving zero margin: the transient memory spikes we documented at 1x are enough to destabilize the eval. Separating the two parameters lets you give containers enough breathing room to avoid spurious OOM kills, while still enforcing a hard ceiling that prevents score

inflation.

鉴于容器运行时实际执行资源的方式——通过保证分配和单独的硬杀阈值——我们建议评估中每个任务指定两个参数，而非单一固定值。单一精确规格将保证分配定为杀死阈值，留零余裕：我们在1倍处记录的瞬态内存尖峰足以使评估不稳定。分开这两个参数可以给容器足够的缓冲空间，避免无谓的 OOM 击杀，同时还能强制执行一个硬性上限，防止分数膨胀。

The band between them should be calibrated so that scores at the floor and ceiling fall within noise of each other. For instance, in Terminal-Bench 2.0, a 3x ceiling over the per-task specs cut infra error rates by roughly two-thirds (5.8% to 2.1%, $p < 0.001$) while keeping the score lift modest and well within noise ($p = 0.40$). That's a reasonable tradeoff: the infrastructure confounder is largely neutralized without removing meaningful resource pressure. The exact multiplier will vary by benchmark and task distribution, and should thus be reported, but the empirical calibration principle is general.

它们之间的分频段应校准，使得底线和天花板的分数处于噪声范围内。例如，在 Terminal-Bench 2.0 中，每个任务规格的 3 倍上限降低了大约三分之二的基础设施错误率（5.8%到 2.1%， $p < 0.001$ ），同时得分提升适中且在噪声范围内（ $p = 0.40$ ）。这是一个合理的权衡：基础设施的混杂因素在没有消除有意义资源压力的情况下基本被中和。具体乘数会因基准和任务分布而异，因此应报告，但经验校准原则是通用的。

Why we care 我们关心的理由

These findings have practical consequences beyond eval infrastructure. Benchmark scores are increasingly used as decision-making inputs, but this increased attention (and reliance) hasn't always come with corresponding rigor in how they're run or reported. As things stand today, a 2-point lead on a leaderboard might reflect a genuine capability difference, or it might reflect that one eval ran on beefier hardware, or even at a luckier time of day, or both. Without published (or standardized) setup configurations, it's hard to tell from the outside unless interested parties go the extra mile to reproduce objective results under identical conditions.

这些发现不仅仅在评估基础设施方面产生了实际影响。基准分数越来越多地被用作决策输入，但这种关注度的增加（和依赖）并不总是伴随着基准分数的运行或报告方式上的严谨性。以目前的情况来看，排行榜上的2分领先可能反映了实际的能力差异，也可能反映某个评估使用了更强大的硬件，甚至是在更幸运的时间，甚至两者兼有。没有公开（或标准化）的设置配置，外部很难判断，除非有兴趣方在相同条件下多走一会儿再现客观结果。

For labs like Anthropic, the implication is that resource configuration for agentic evals should be treated as a first-class experimental variable, documented and controlled with the same rigor as prompt format or sampling temperature. For benchmark maintainers, publishing recommended resource specs (as Terminal-Bench 2.0 does) can go a long way, while specifying enforcement methodology would close the gap we identified. And for anyone consuming benchmark results, the core takeaway is that small score differences on agentic evals carry more uncertainty than the precision of the reported numbers suggests—especially as some confounders are simply too hard to control for.

对于像 Anthropic 这样的实验室来说，这意味着代理评估的资源配置应被视为一流的实验变量，以与提示格式或采样温度同等严谨的记录和控制。对于基准维护者来说，发布推荐的资源规格（如 Terminal-Bench 2.0 所做的）可以发挥很大作用，而规范执行方法则能弥合我们发现的差距。对于任何关注基准结果的人来说，核心结论是，代理性评估中的微小分数差异比报告数据的精确度所显示的更不确定——尤其是某些混杂因素难以控制。

Until resource methodology is standardized, our data suggests that leaderboard differences below 3 percentage points deserve skepticism until the eval configuration is documented and matched. The observed spread across the moderate range of resource configurations in Terminal-Bench is just below 2 percentage points. Naive binomial confidence intervals already span 1-2 percentage points; the infrastructure confounders we document here stack on top of that, not within it. At the extremes of the allocation range, the spread reaches 6.

在资源方法标准化之前，我们的数据表明，在评估配置被记录并匹配之前，领先排名差异低于3个百分点的差异应予以怀疑。终端-工作台中观察到的中等资源配置区间的分布略低于2个百分点。朴素的二项置信区间已经跨越1-2个百分点；我们在这里记录的基础设施混杂因素是叠加在这些基础之上，而不是在它内部。在配置区间的极端位置，利差达到6。

A few-point lead might signal a real capability gap—or it might just be a bigger VM. 几点的领先可能表明存在真正的能力差距——也可能只是虚拟机更大。

Acknowledgements 致谢

Written by Gian Segato. Special thanks to Nicholas Carlini, Jeremy Hadfield, Mike Merrill, and Alex Shaw for their contributions. This work reflects the collective efforts of several teams working on evaluations for coding agents. Interested candidates who would like to contribute are welcome to apply at

anthropic.com/careers.

作者：吉安·塞加托。特别感谢尼古拉斯·卡里尼、杰里米·哈德菲尔德、迈克·梅里尔和亚历克斯·肖的贡献。这项工作反映了多个团队共同努力，致力于编码代理的评估。有意愿贡献的候选人欢迎在 anthropic.com/careers 申请。

Get the developer newsletter

订阅开发者通讯

Product updates, how-tos, community spotlights, and more.

Delivered monthly to your inbox.

产品更新、操作指南、社区聚焦灯光，以及更多内容。每月发送至您的邮箱。



Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time.

如果你想收到我们的月度开发者通讯，请提供你的电子邮箱。您可以随时取消订阅。



Products

Claude

Claude Code

Claude Code Enterprise

Claude Cowork

Claude for Chrome

Claude for Excel

Claude for PowerPoint

Claude for Slack

Skills

Max plan

Team plan

Enterprise plan

Download app

Pricing

Log in to Claude

Models

Opus

Sonnet

Haiku

Solutions

AI agents

Claude Code Security

Code modernization

Coding

Customer support

Education

Financial services

Government

Healthcare

Life sciences

Nonprofits

Claude Platform

Overview

Developer docs

Pricing

Marketplace

Regional compliance

Amazon Bedrock

Google Cloud's Vertex AI

Microsoft Foundry

Console login

Resources

Blog

Claude partner network

Community

Connectors

Courses

Customer stories

Engineering at Anthropic

Events

Inside Cowork

Plugins

Powered by Claude

Service partners

Startups program

Tutorials

Use cases

Company

Anthropic

Careers

Economic Futures

Research

News

Claude's Constitution

Responsible Scaling Policy

Security and compliance

Transparency

Help and security

Availability

Status

Support center

Terms and policies

Privacy choices

Privacy policy

Consumer health data privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy

© 2026 Anthropic PBC

